

Comparative Analysis of Lion and AdamW Optimizers for Cross-Encoder Reranking with MiniLM, GTE, and ModernBERT

Shahil Kumar
Department of IT
IIIT Allahabad
Prayagraj, India
mml2023008@iiita.ac.in

Manu Pande
Department of IT
IIIT Allahabad
Prayagraj, India
mml2023005@iiita.ac.in

Anay Yatin Damle
Department of IT
IIIT Allahabad
Prayagraj, India
mml2023016@iiita.ac.in

Abstract—Modern information retrieval systems often employ a two-stage pipeline consisting of an efficient initial retrieval stage followed by a more computationally intensive reranking stage. Cross-encoder models have demonstrated state-of-the-art effectiveness for the reranking task due to their ability to perform deep, contextualized analysis of query-document pairs. The choice of optimizer during the fine-tuning phase can significantly impact the final performance and training efficiency of these models. This paper investigates the impact of using the recently proposed Lion optimizer compared to the widely used AdamW optimizer for fine-tuning cross-encoder rerankers. We fine-tune three distinct transformer models, ‘microsoft/MiniLM-L12-H384-uncased’, ‘Alibaba-NLP/gte-multilingual-base’, and ‘answerdotai/ModernBERT-base’, on the MS MARCO passage ranking dataset using both optimizers. Notably, GTE and ModernBERT support longer context lengths (8192 tokens). The effectiveness of the resulting models is evaluated on the TREC 2019 Deep Learning Track passage ranking task and the MS MARCO development set (for MRR@10). Our experiments, facilitated by the Modal cloud computing platform for GPU resource management, show comparative results across three training epochs. ModernBERT trained with Lion achieved the highest NDCG@10 (0.7225) and MAP (0.5121) on TREC DL 2019, while MiniLM trained with Lion tied with ModernBERT with Lion on MRR@10 (0.5988) on MS MARCO dev. Additionally, Lion optimizer demonstrates superior GPU utilization efficiency across all models, achieving efficiency gains of 2.67% to 10.33% while maintaining competitive performance. We analyze the performance trends based on standard IR metrics, providing insights into the relative effectiveness of Lion versus AdamW for different model architectures and training configurations in the context of passage reranking. The code for training and evaluation is available at <https://github.com/skfrost19/Cross-Encoder-Lion-vs-AdamW>, and the trained models are available on Huggingface Model Hub <https://huggingface.co/collections/skfrost19/rerankers-681320776cfb45e44b18f5f1>.

Index Terms—Information Retrieval, Cross-Encoder, Reranking, Optimizer, Lion Optimizer, AdamW, TREC Deep Learning, MS MARCO, Sentence Transformers, ModernBERT, Long Context, Modal.

I. INTRODUCTION

Modern Information Retrieval (IR) systems employ two-stage retrieve-and-rerank pipelines [1], where efficient first-stage methods (BM25 [2], dense retrieval [3]) retrieve candi-

date documents, followed by sophisticated reranking models for precision optimization. Cross-encoder models based on transformer architectures [4] have achieved state-of-the-art reranking performance [5], [6] through simultaneous query-document processing ([CLS] query [SEP] document [SEP]), enabling deep token-level interactions despite higher computational costs. Modern models like GTE [7] and ModernBERT [8] support extended context lengths (8192 tokens) compared to efficiency-focused models like MiniLM [9].

Optimizer selection critically impacts transformer fine-tuning effectiveness. While AdamW [10] remains widely adopted, the recently proposed Lion optimizer [11] (EvoLved Sign Momentum), derived through symbolic mathematics, demonstrates improved performance and memory efficiency in vision tasks.

This work investigates Lion’s effectiveness for cross-encoder passage reranking compared to AdamW across three transformer models with varying characteristics. Our contributions are:

- We fine-tune three distinct transformer-based cross-encoder models (‘microsoft/MiniLM-L12-H384-uncased’, ‘Alibaba-NLP/gte-multilingual-base’, and ‘answerdotai/ModernBERT-base’) on the large-scale MS MARCO passage dataset [12] using both Lion and AdamW optimizers.
- We utilize different training configurations optimized for specific models, including a lower learning rate (2e-6) and a Cosine Annealing scheduler for ModernBERT, contrasting with a higher rate (2e-5) and no scheduler for MiniLM and GTE.
- We evaluate the performance of the fine-tuned models across three training epochs on the TREC 2019 Deep Learning (DL) Track [13] benchmark and the MS MARCO development set [12].
- We provide a comparative analysis of the optimizers’ impact on reranking effectiveness using standard IR metrics (NDCG@10, MAP, MRR@10, Recall@10, R-Prec, P@10).
- We demonstrate that Lion optimizer achieves superior

GPU utilization efficiency compared to AdamW across all tested models, with efficiency gains ranging from 2.67% to 10.33%, while maintaining competitive or superior performance metrics.

- We utilize the Modal cloud platform [14] for efficient GPU resource management (3x NVIDIA L40S-48GB) and reproducible experimentation.

The rest of the paper is organized as follows: Section II discusses related work. Section III details the models, optimizers, and training approach. Section IV describes the experimental setup, datasets, and evaluation protocol. Section V presents and discusses the results. Finally, Section VI concludes the paper and suggests future work.

II. RELATED WORK

A. Cross-Encoder Reranking

BERT-based cross-encoders for document reranking were pioneered by Nogueira et al. [5], [6], demonstrating superior performance over traditional IR models and bi-encoders through joint query-document processing. Subsequent advances explored diverse architectures [15], training strategies [16], and efficiency improvements [17]. Modern developments include efficiency-focused models like MiniLM [9] (knowledge distillation), multilingual models like GTE [7] supporting longer contexts, and architecturally enhanced models like ModernBERT [8] incorporating RoPE [18] and Flash Attention [19] for 8192-token sequences. The ‘sentence-transformers’ library [20] provides standardized frameworks for cross-encoder training and deployment.

B. Optimizers for Deep Learning

While SGD with momentum remains fundamental, adaptive methods like Adam [21] combining momentum with adaptive learning rates dominate deep learning optimization. AdamW [10] improves Adam by decoupling weight decay from adaptive mechanisms, enhancing generalization. The Lion optimizer [11] employs a simplified approach using momentum tracking and sign operations (‘update = sign(momentum) * lr’), requiring less memory by avoiding second moment estimates. Lion’s effectiveness, primarily demonstrated in vision and vision-language tasks, motivates its evaluation in NLP/IR domains.

C. Evaluation Benchmarks

MS MARCO [12] serves as the standard training benchmark for passage retrieval models, containing real Bing queries with human-judged relevance. TREC Deep Learning Tracks [13], [22] provide challenging test collections with sparse relevance judgments from pooled system results. We evaluate on TREC DL 2019 passage ranking using ‘trec_eval’ [23] metrics and MS MARCO development set for MRR@10, providing comprehensive effectiveness assessment across different query scales.

III. METHODOLOGY

A. Cross-Encoder Architecture

A cross-encoder model takes a query q and a document d as input, typically concatenating them with special tokens: ‘[CLS] q [SEP] d [SEP]’. This combined sequence is fed into a transformer model (e.g., MiniLM, GTE, ModernBERT). The output representation corresponding to the ‘[CLS]’ token is then passed through a linear layer followed by a sigmoid activation to predict a relevance score $s(q, d) \in [0, 1]$. During inference for reranking, this score is computed for all candidate documents retrieved in the first stage, and the documents are re-sorted based on these scores. An overview of the architecture is shown in Fig. 1.

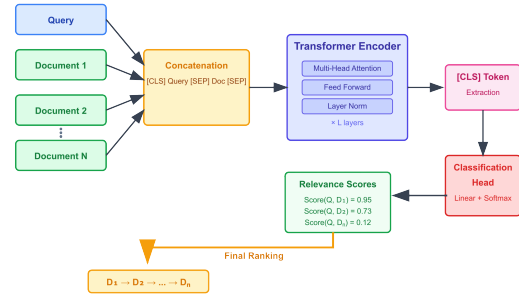


Fig. 1: General Cross-Encoder Architecture for Reranking.

B. Base Models

We experiment with three transformer base models from Hugging Face [24]:

- **‘microsoft/MiniLM-L12-H384-uncased’ [9]:** A distilled version of BERT, designed to be smaller (12 layers, 384 hidden size) and faster while retaining significant performance. It has a standard context length, typically 512 tokens.
- **‘Alibaba-NLP/gte-multilingual-base’ [7]:** Part of the General Text Embeddings (GTE) family, trained on a large, diverse multilingual corpus. While often used as a bi-encoder, its underlying transformer architecture can be effectively fine-tuned as a cross-encoder. This ‘base’ version supports a context length of 8192 tokens.
- **‘answerdotai/ModernBERT-base’ [8]:** A state-of-the-art encoder-only transformer incorporating architectural enhancements like Rotary Positional Embeddings (RoPE) [18] for effective long context handling (up to 8192 tokens), Flash Attention [19] for speed and memory efficiency, and GeGLU activation functions [25]. It was trained on 2 trillion tokens and demonstrates strong performance on various tasks.

C. Training

We fine-tune the cross-encoders using the MS MARCO passage ranking triplets dataset [12], as processed by ‘sentence-transformers’ [20]. The original dataset contains tuples of (query, positive passage, negative passage). We convert this into pairs ‘(query, positive_passage)’ with label 1 and ‘(query, negative_passage)’ with label 0. We train on approximately 2 million such pairs. The training objective is to minimize the Binary Cross-Entropy (BCE) loss between the predicted relevance score $\hat{y}$ (output of the sigmoid function) and the true label y (0 or 1). The BCE loss is defined as:

$$\mathcal{L}_{BCE} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

D. Optimizers

We compare two optimizers using specified parameters:

- **AdamW [10]:** Implemented via `torch.optim.AdamW`. We use a weight decay of 0.01. The learning rate varies by model (see Section IV).
- **Lion [11]:** Implemented via a PyTorch implementation based on the original paper. We use the default betas $\beta_1 = 0.9$, $\beta_2 = 0.99$, and a weight decay of 0.01. The learning rate varies by model. The update rule is as follows:

$$\begin{aligned} c_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ \theta_t &= \theta_{t-1} - \eta (\text{sign}(c_t) + \lambda \theta_{t-1}) \\ m_t &= \beta_2 m_{t-1} + (1 - \beta_2) g_t \end{aligned}$$

where g_t is the gradient, m_t is the momentum, η is the learning rate, β_1, β_2 are momentum coefficients, and λ is the weight decay.

IV. EXPERIMENTAL SETUP

A. Datasets

- **Training:** We use the MS MARCO passage ranking triplets dataset [12], processed into approximately 2 million query-passage pairs with binary labels using the ‘sentence-transformers’ methodology.
- **Evaluation (Main):** We evaluate on the TREC 2019 Deep Learning Track passage ranking task [13], which contains 43 queries with graded relevance judgments (qrels).
- **Evaluation (MRR@10):** We evaluate Mean Reciprocal Rank at cutoff 10 (MRR@10) on the MS MARCO passage dataset’s development split (available from Hugging Face Datasets [12]), containing a larger set of queries with binary relevance.

B. Implementation Details

- **Framework:** We use ‘sentence-transformers’ [20] (‘CrossEncoder’) built on PyTorch [26] and Hugging Face Transformers [24].
- **Hyperparameters:** Key training parameters are:
 - Batch Size: 64.
 - Learning Rate:

- * 2e-5 for MiniLM and GTE (both optimizers).
- * 2e-6 for ModernBERT with Lion optimizer.
- * 2e-5 for ModernBERT with adamW optimizer.

– Scheduler:

- * None for MiniLM and GTE.
- * CosineAnnealingLR Scheduler for ModernBERT, with ‘T_max’ set to the total number of training steps.

– Optimizer Params:

- * AdamW: weight_decay=0.01.
- * Lion: betas=(0.9, 0.99), weight_decay=0.01.

– Warmup Ratio: 0.1 (for LR scheduler if used)

– Epochs: 3

– Precision: BF16 enabled.

– Seed: 12 (for reproducibility).

– Dataloader Workers: 4.

• Infrastructure:

- Experiments were conducted using the Modal platform [14], leveraging 3x NVIDIA L40S-48GB GPUs. Modal managed the environment (CUDA, Python 3.11, libraries) and distributed training.
- Experiment tracking, visualization of training metrics (such as loss and learning rate), and hyperparameter logging were managed using the Weights & Biases (W&B) platform [27].

- **First-Stage Retrieval:** For TREC DL evaluation, we retrieve the top 1000 candidate passages per query using a standard BM25 baseline via Pyserini [28] on the ‘msmarco-v1-passage’ index. The cross-encoders rerank this candidate set.

C. Evaluation Protocol

- **Reranking:** Each fine-tuned cross-encoder model scores the top-1000 BM25 candidates for each TREC DL 2019 query. Passages are reranked based on these scores.
- **Metrics:** We use ‘trec_eval’ [23] for TREC DL 2019 evaluation. We report:
 - NDCG@10 (Normalized Discounted Cumulative Gain @10)
 - MAP (Mean Average Precision)
 - Recall@10 (Recall @10)
 - R-Prec (R-Precision)
 - P@10 (Precision @10)

For MRR@10, we evaluate on the MS MARCO development set.

- **Configurations Tested:** We evaluate each combination of base model and optimizer after each of the 3 training epochs:

- MiniLM + AdamW (Epochs 1, 2, 3)
- MiniLM + Lion (Epochs 1, 2, 3)
- GTE-multilingual-base + AdamW (Epochs 1, 2, 3)
- GTE-multilingual-base + Lion (Epochs 1, 2, 3)
- ModernBERT-base + AdamW (Epochs 1, 2, 3)
- ModernBERT-base + Lion (Epochs 1, 2, 3)

V. RESULTS AND DISCUSSION

Table I presents the evaluation results for all configurations on the TREC 2019 Deep Learning Track (NDCG@10, MAP, Recall@10, R-Prec, P@10) and MS MARCO development set (MRR@10) after 1, 2, and 3 epochs of fine-tuning. Best results for each metric are highlighted.

A. GPU Efficiency Analysis

Beyond performance metrics, we analyze the GPU resource efficiency of both optimizers during training. Table II presents detailed GPU utilization statistics collected across 3 NVIDIA L40S-48GB GPUs during the training process.

The results demonstrate that Lion optimizer consistently achieves better GPU utilization efficiency compared to AdamW across all tested models:

- **MiniLM-L12-H384:** Lion shows a modest efficiency improvement of 2.67%, reducing mean GPU usage from 33.09% to 32.21% and peak usage from 35.25% to 34.64%.
- **GTE-multilingual-base:** The most significant efficiency gain of 10.33% is observed with Lion, reducing mean GPU usage from 73.04% to 65.50% and peak usage from 78.08% to 69.69%.
- **ModernBERT-base:** Lion achieves a 3.49% efficiency improvement, reducing mean GPU usage from 77.04% to 74.35% and peak usage from 81.40% to 79.45%.

These efficiency gains are particularly notable given that Lion maintains competitive or superior performance metrics (as shown in Table I). The lower GPU utilization suggests that Lion’s simpler update rule, which only requires tracking momentum and does not store second moment estimates like AdamW, translates to practical computational savings during training. This aligns with the theoretical memory efficiency advantages of Lion optimizer reported in the original paper [11].

The largest efficiency gain observed with GTE-multilingual-base (10.33%) is especially significant considering this model’s larger size and 8K context length capability. This suggests that Lion’s benefits may be more pronounced for larger, more computationally intensive models.

B. Optimizer Performance Comparison

We observe different interactions between the base models and the optimizers:

- **MiniLM:** AdamW generally achieves higher peak performance on TREC DL 2019 metrics (NDCG@10: 0.7127 vs 0.7031; MAP: 0.4908 vs 0.4858), peaking at Epoch 3. Lion achieves the overall best MRR@10 on MS MARCO dev (0.5988 at Epoch 3), but its TREC performance peaks earlier (Epoch 1) and then declines. This suggests AdamW might be more stable over longer training for MiniLM on TREC, while Lion might converge faster or optimize differently for the MS MARCO distribution.
- **GTE:** AdamW clearly outperforms Lion for the GTE model across most metrics and epochs.

GTE+AdamW achieves its best TREC performance (NDCG@10=0.7224, MAP=0.5005) early at Epoch 1 or 2, with performance degrading by Epoch 3. GTE+Lion shows weaker performance overall, peaking later (Epoch 2/3) but never reaching the levels of GTE+AdamW.

- **ModernBERT:** Lion significantly outperforms AdamW when training ModernBERT with the specific low learning rate (2e-6) and Cosine Annealing scheduler. ModernBERT+Lion achieves the overall best NDCG@10 (0.7225), MAP (0.5121), R-Prec (0.5183), and P@10 (0.8256) in our experiments. Its performance peaks around Epoch 2 for most TREC metrics. ModernBERT+AdamW, using the same LR and scheduler, performed notably worse, suggesting Lion interacts more favorably with this setup for this model.

These results indicate that the choice of optimizer interacts significantly with the base model architecture and the specific training hyperparameters (learning rate, scheduler). Lion showed particular strength with ModernBERT under its tailored training regime.

C. Model Performance Comparison

Comparing the best results achieved by each model (irrespective of optimizer epoch):

- **ModernBERT (+Lion, E2/E1/E3):** Achieved the highest NDCG@10 (0.7225), MAP (0.5121), R-Prec (0.5183), and P@10 (0.8256) on TREC DL 2019, and tied for the best MRR@10 (0.5988) on MS MARCO dev. This suggests its advanced architecture and long context, combined with the Lion optimizer and specific training strategy, yield superior reranking quality.
- **GTE (+AdamW, E1/E2):** Achieved the next best performance, with NDCG@10 (0.7224) nearly matching ModernBERT+Lion, and achieving the best Recall@10 (0.1733). Its peak performance was reached earlier than ModernBERT’s in terms of epochs.
- **MiniLM (+AdamW E3 / +Lion E3):** While performing competitively, MiniLM generally lagged behind GTE and ModernBERT on TREC metrics. However, MiniLM+Lion achieved the joint-highest MRR@10 (0.5988) on the MS MARCO dev set, indicating strong performance on that specific benchmark despite lower TREC scores compared to the larger models.

The results suggest that the larger models with longer context capabilities (GTE, ModernBERT) generally outperform MiniLM on the TREC DL 2019 task. ModernBERT, with its specific optimizations and training configuration using Lion, achieved the overall best effectiveness.

D. Performance Trends Over Epochs and Training Dynamics

Observing Table I, performance is not always monotonic with training epochs.

- GTE+AdamW peaks early (Epoch 1 or 2) and then degrades.
- MiniLM+AdamW shows more gradual improvement or stability up to Epoch 3.

TABLE I: Evaluation Results on TREC-DL 2019 and MS-MARCO Dev Passage Ranking

Base Model	Optimizer	Epoch	NDCG@10	MAP	MRR@10	Recall@10	R-Prec	P@10
MiniLM-L12-H384	AdamW	1	0.7008	0.4814	0.5828	0.1712	0.4899	0.8047
		2	0.7094	0.4891	0.5818	0.1715	0.5017	0.8093
		3	0.7127	0.4908	0.5826	0.1706	0.4962	0.8023
MiniLM-L12-H384	Lion	1	0.7031	0.4858	0.5890	0.1698	0.4904	0.8070
		2	0.6916	0.4755	0.5942	0.1724	0.5041	0.8116
		3	0.6808	0.4706	0.5988	0.1701	0.4923	0.8023
GTE-multilingual-base	AdamW	1	0.7224	0.5005	0.5940	0.1733	0.4957	0.8140
		2	0.7203	0.4999	0.5942	0.1733	0.5067	0.8163
		3	0.6902	0.4899	0.5972	0.1730	0.5069	0.8140
GTE-multilingual-base	Lion	1	0.6785	0.4754	0.5854	0.1684	0.4849	0.7953
		2	0.6909	0.4921	0.5957	0.1721	0.5053	0.8140
		3	0.6904	0.4912	0.5931	0.1719	0.5041	0.8093
Modern-BERT-base	AdamW	1	0.7105	0.5066	0.5865	0.1678	0.5161	0.8163
		2	0.6839	0.4893	0.5885	0.1634	0.4946	0.7814
		3	0.6959	0.4971	0.5916	0.1623	0.5116	0.7860
Modern-BERT-base	Lion	1	0.7142	0.5121	0.5834	0.1689	0.5148	0.8163
		2	0.7225	0.5115	0.5907	0.1732	0.5183	0.8209
		3	0.7051	0.5020	0.5988*	0.1722	0.5102	0.8256

* MRR@10 is calculated on the MS MARCO v1.1 passage dataset development split. All other metrics are calculated on TREC DL 2019. Best result for each metric highlighted. (*Note: MiniLM+Lion also achieved 0.5988 MRR@10 at epoch 3).

TABLE II: GPU Usage Statistics Comparison: Lion vs AdamW Optimizers

Model	Optimizer	Mean Usage (%)	Peak Usage (%)	Std Dev (%)	Data Points	Efficiency Gain (%)
MiniLM-L12-H384	AdamW	33.09	35.25	2.72	410	+2.67%
	Lion	32.21	34.64	2.78	412	
GTE-multilingual-base	AdamW	73.04	78.08	7.05	698	+10.33%
	Lion	65.50	69.69	6.27	699	
ModernBERT-base	AdamW	77.04	81.40	6.01	578	+3.49%
	Lion	74.35	79.45	8.97	579	

GPU utilization measured across 3 NVIDIA L40S-48GB GPUs during training. Efficiency gain calculated as: $\frac{\text{AdamW Mean} - \text{Lion Mean}}{\text{AdamW Mean}} \times 100\%$. Lower usage indicates better efficiency.

- MiniLM+Lion peaks early on TREC (Epoch 1) but late on MS MARCO MRR (Epoch 3).
- ModernBERT+Lion mostly peaks around Epoch 2.
- ModernBERT+AdamW performance is less consistent, potentially indicating suboptimal interaction with the LR/scheduler setup.

This highlights the importance of evaluating at multiple checkpoints. Additionally, analysis of training/evaluation loss curves, learning rate schedules, and gradient norms for ModernBERT provides further insight into the training dynamics, showing convergence patterns under the Cosine Annealing schedule with both optimizers.

E. Comparison with State-of-the-Art

Table III situates our best result (ModernBERT + Lion, Epoch 2) relative to other published results on TREC DL 2019 passage ranking.

Our best model, ModernBERT+Lion (NDCG@10 = 0.7225), achieves the highest performance among the models listed in Table III, surpassing strong baselines such as BM25 (0.506), dense retrieval models like ANCE (0.648) and BM25 Neg (0.664), hybrid models such as Bi-encoder

(PoolAvg) + doc2query-T5 (0.719), and cross-encoder models including cross-encoder/ms-marco-electra-base (0.7199). Our GTE+AdamW model achieves nearly identical performance (0.7224), while our MiniLM+AdamW variant scores 0.7127, remaining competitive. These results highlight the effectiveness of our optimizer choices and training strategies for base-sized models.

VI. CONCLUSION

In this paper, we presented a comparative study of the Lion and AdamW optimizers for fine-tuning three different transformer models ('microsoft/MiniLM-L12-H384-uncased', 'Alibaba-NLP/gte-multilingual-base', 'answerdotai/ModernBERT-base') as cross-encoders for passage reranking. Training was performed on MS MARCO, and evaluation used TREC 2019 Deep Learning Track and MS MARCO dev benchmarks. Our findings indicate:

- The choice of optimizer significantly interacts with the base model and training hyperparameters (LR, scheduler).
- For ModernBERT, using a low learning rate (2e-6) and Cosine Annealing scheduler, Lion substantially outperformed AdamW, achieving the best overall TREC DL

TABLE III: Comparison with Other Systems on TREC DL 2019 (NDCG@10)

Model/System	NDCG@10	Reference
Sparse Retrieval (single stage)		
BM25	0.506	[29]
DeepCT	0.551	[16]
doc2query-T5	0.642	[15]
Dense Retrieval Models		
ANCE	0.648	[30]
Rand Neg	0.605	[30]
NCE Neg	0.602	[30]
BM25 Neg	0.664	[30]
DPR (BM25 + Rand Neg)	0.653	[30]
BM25 → Rand	0.609	[30]
BM25 → NCE Neg	0.608	[30]
BM25 → BM25 + Rand	0.648	[30]
Hybrid dense + sparse (single stage)		
Bi-encoder (PoolAvg) + BM25	0.701	[29]
Bi-encoder (TCT-ColBERT) + BM25	0.714	[29]
Bi-encoder (PoolAvg) + doc2query-T5	0.719	[29]
Cross-Encoder/Reranking Models		
BERT-base Cross-Encoder	0.634	[15]
BERT-large Cross-Encoder	0.654	[15]
pt-bert-base-uncased-msmarco	0.709	[31]
cross-encoder/ms-marco-electra-base	0.719	[20]
cross-encoder/ms-marco-TinyBERT-L-2-v2	0.698	[20]
Our Best (ModernBERT+Lion E2)	0.7225	This work
Our GTE+AdamW E1/E2	0.7224 / 0.7203	This work
Our MiniLM+AdamW E3	0.7127	This work

NDCG@10 scores are reported as evaluated on the TREC DL 2019 dataset using standard settings from cited references. Minor variations may occur due to differences in implementation, random seeds, or reranking candidate sets.

2019 results (NDCG@10 = 0.7225, MAP = 0.5121 at Epochs 1-2).

- For GTE, AdamW (with LR 2e-5, no scheduler) was more effective than Lion, achieving strong results (NDCG@10 = 0.7224) comparable to ModernBERT+Lion.
- For MiniLM, AdamW yielded slightly better TREC performance over 3 epochs, while Lion achieved the best MS MARCO dev MRR@10 (tied with ModernBERT+Lion).
- Models with longer context capabilities (GTE, ModernBERT) generally outperformed MiniLM on the TREC DL task.
- Performance varied across epochs, emphasizing the need for checkpoint selection based on validation performance.
- **Lion optimizer demonstrates superior GPU utilization efficiency across all models**, with efficiency gains ranging from 2.67% (MiniLM) to 10.33% (GTE), while maintaining competitive or superior performance metrics.

Our results show that Lion can be a highly effective optimizer for cross-encoder training, particularly for newer architectures like ModernBERT when paired with appropriate learning rate strategies. The combination of competitive performance and improved resource efficiency makes Lion an attractive choice

for practical applications. However, AdamW remains a strong baseline, especially for models like GTE in this setup. The Modal platform proved effective for managing the required GPU resources and experimental setup.

Future work could involve a more thorough hyperparameter search for both optimizers across all models, particularly exploring different learning rates and schedules for Lion with MiniLM and GTE. Investigating the impact of the 8K context length more directly (e.g., on datasets with longer documents) and conducting more detailed analysis of memory usage differences between optimizers would also be valuable. Additionally, the observed GPU efficiency gains with Lion optimizer warrant further investigation across different hardware configurations and model scales to understand the broader implications for large-scale training deployments.

ACKNOWLEDGMENT

The authors acknowledge Modal Labs (<https://modal.com/>) for providing the cloud computing platform and GPU(s) resources (3x NVIDIA L40S-48GB) used for conducting the experiments presented in this paper.

REFERENCES

- [1] M. Hu, Y. Peng, Z. Huang, and D. Li, "Retrieve, read, rerank: Towards end-to-end multi-document reading comprehension," 2019. [Online]. Available: <https://arxiv.org/abs/1906.04618>
- [2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends® in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [3] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186. [Online]. Available: <https://www.aclweb.org/anthology/N19-1423>
- [5] R. Nogueira and K. Cho, "Passage re-ranking with bert," 2020. [Online]. Available: <https://arxiv.org/abs/1901.04085>
- [6] R. Nogueira, Z. Yang, K. Cho, and J. Lin, "Document ranking with a pretrained sequence-to-sequence model," in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, Nov. 2020, pp. 1124–1136. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.102>
- [7] Z. Li, X. Zhang, Y. Zhang, D. Long, P. Xie, and M. Zhang, "Towards general text embeddings with multi-stage contrastive learning," 2023. [Online]. Available: <https://arxiv.org/abs/2308.03281>
- [8] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini, A. Gallagher, R. Biswas, F. Ladhak, T. Aarsen, N. Cooper, G. Adams, J. Howard, and I. Poli, "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," 2024. [Online]. Available: <https://arxiv.org/abs/2412.13663>
- [9] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," 2020. [Online]. Available: <https://arxiv.org/abs/2002.10957>
- [10] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019. [Online]. Available: <https://arxiv.org/abs/1711.05101>
- [11] X. Chen, C. Liang, D. Huang, E. Real, K. Wang, Y. Liu, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu, and Q. V. Le, "Symbolic discovery of optimization algorithms," 2023. [Online]. Available: <https://arxiv.org/abs/2302.06675>

- [12] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, "MS MARCO: A human generated machine reading comprehension dataset," *CoRR*, vol. abs/1611.09268, 2016. [Online]. Available: <http://arxiv.org/abs/1611.09268>
- [13] N. Craswell, B. Mitra, E. Yilmaz, D. Campos, and E. M. Voorhees, "Overview of the trec 2019 deep learning track," 2020. [Online]. Available: <https://arxiv.org/abs/2003.07820>
- [14] Modal, "Modal: Serverless compute for ai and data workflows," <https://modal.com/>, 2025, accessed: 20-04-2025.
- [15] J. Lin, R. Nogueira, and A. Yates, "Pretrained transformers for text ranking: Bert and beyond," in *Synthesis Lectures on Human Language Technologies*, 2021. [Online]. Available: <https://arxiv.org/abs/2010.06467>
- [16] L. Gao, Z. Dai, T. Chen, Z. Fan, B. V. Durme, and J. Callan, "Complementing lexical retrieval with semantic residual embedding," 2021. [Online]. Available: <https://arxiv.org/abs/2004.13969>
- [17] S. Hofstätter, S. Althammer, M. Schröder, M. Sertkan, and A. Hanbury, "Improving efficient neural ranking models with cross-architecture knowledge distillation," 2021. [Online]. Available: <https://arxiv.org/abs/2010.02666>
- [18] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," 2023. [Online]. Available: <https://arxiv.org/abs/2104.09864>
- [19] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [20] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 11 2019, pp. 3982–3992. [Online]. Available: <https://arxiv.org/abs/1908.10084>
- [21] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [22] N. Craswell, B. Mitra, E. Yilmaz, and D. Campos, "Overview of the trec 2020 deep learning track," 2021. [Online]. Available: <https://arxiv.org/abs/2102.07662>
- [23] National Institute of Standards and Technology (NIST), "trec_eval information retrieval evaluation software," https://github.com/usnistgov/trec_eval, 2024.
- [24] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. [Online]. Available: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [25] N. Shazeer, "Glu variants improve transformer," 2020. [Online]. Available: <https://arxiv.org/abs/2002.05202>
- [26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>
- [27] L. Biewald, "Experiment tracking with weights and biases," 2020, software available from wandb.com. [Online]. Available: <https://www.wandb.com/>
- [28] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, "Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 2356–2360. [Online]. Available: <https://dl.acm.org/doi/10.1145/3404835.3463238>
- [29] S.-C. Lin, J.-H. Yang, and J. Lin, "Distilling dense representations for ranking using tightly-coupled teachers," 2020. [Online]. Available: <https://arxiv.org/abs/2010.11386>
- [30] L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. Bennett, J. Ahmed, and A. Overwijk, "Approximate nearest neighbor negative contrastive learning for dense text retrieval," 2020. [Online]. Available: <https://arxiv.org/abs/2007.00808>
- [31] C. Thienes and J. Pertschuk, "Nboost: Neural boosting search results," <https://github.com/koursaros-ai/nboost>, 2019.